DATA ORIGIN AND REPRODUCIBILITY

Where the Logic Engine data comes from

Plain-language provenance overview by Glyph Systems

Estimated reading time: 8-10 minutes

INVESTOR • CUSTOMER • STRATEGIC PARTNER OVERVIEW

July 22, 2026

This overview explains what data was public, what data was modeled, what a seed means, what was frozen before a run, and how a reviewer can trace a result back to its source.

Data Origin and Reproducibility

This document explains where the Logic Engine test data came from, what was public, what was modeled, what a seed means, what was frozen before a run, and how a reviewer can trace a result back to its source.

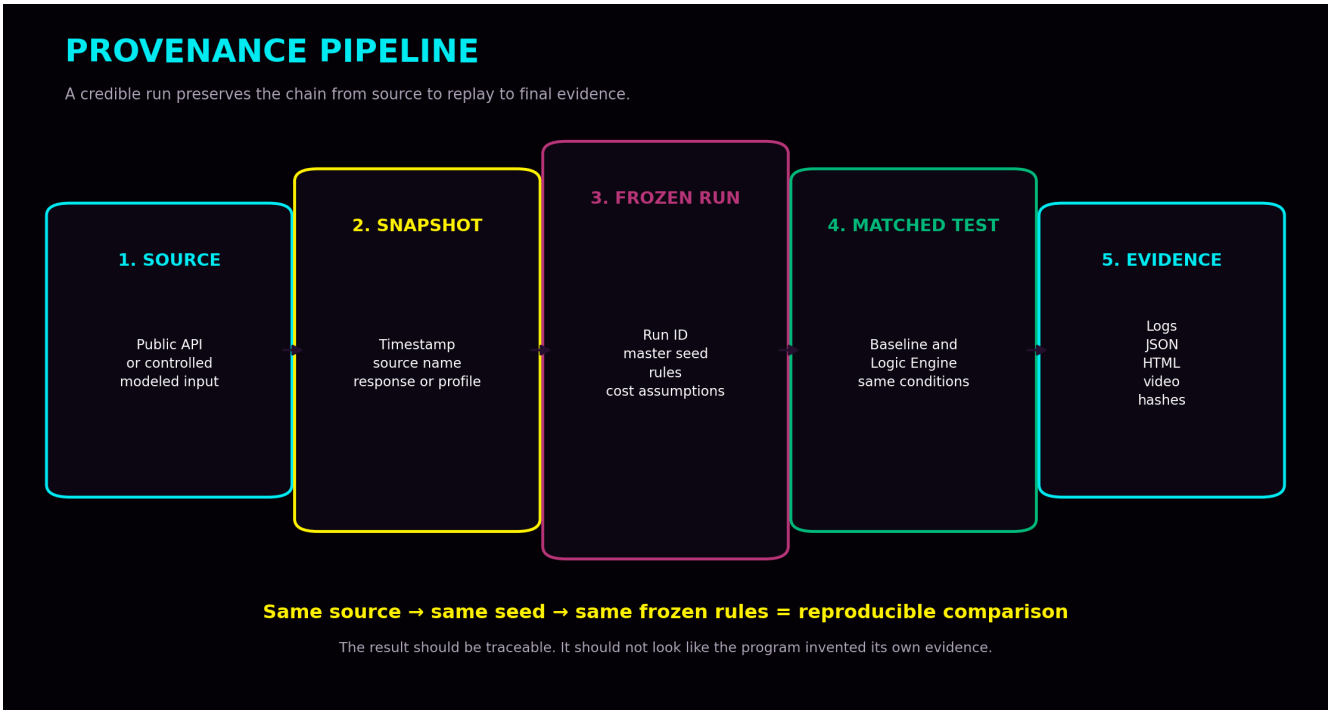
The goal is simple: the result should be traceable. It should not look like the program invented its own evidence.

> **Plain-language rule: every serious run should show the source, the snapshot, the seed, the frozen rules, and the preserved outputs.**

1. Why data origin matters

If a system claims a result but cannot show where the information came from, a serious investor, auditor, or customer should distrust it. The same applies if the rules change after the result is known.

- A reviewer must be able to tell what information was observed.
- A reviewer must be able to tell what information was modeled.
- A reviewer must be able to see what was frozen before the run began.
- A reviewer must be able to inspect the logs, reports, and evidence package afterward.



2. The three data types

Data type	Meaning
Live public data	Outside-world information pulled from public sources such as Bitcoin network APIs or official service-status feeds.

Data type	Meaning
Controlled modeled data	Synthetic or compressed workloads built to represent payment, network, or treasury operating conditions.
Future customer data	Authorized historical or read-only live telemetry supplied by a customer during an approved pilot.

Keeping these categories separate prevents a reader from mistaking a controlled model for a live customer deployment.

3. Public source examples

Several runs relied on public inputs rather than private or hidden sources. Examples include:

- Mempool.space fee and mempool conditions for Bitcoin congestion timing
- CoinGecko price reference data for Bitcoin-denominated examples
- GitHub Status snapshots for public incident profiles
- Cloudflare Status snapshots for public incident profiles

> **Logic Engine did not create the Bitcoin fee conditions or public-incident conditions. It observed public information, preserved it, and used it as an input.**

4. What a seed means

A seed is the recorded starting key used to generate a specific test sequence. Using the same seed recreates the same event order. Using a new seed creates a different—but still controlled—event order.

- The seed supports reproducibility.
- The seed lets Glyph Systems rerun the same test later.
- The seed also lets Glyph Systems test whether a result survives new event sequences.

5. What is frozen before a run

A serious run should not improvise after the score is visible. The following items should be declared and frozen before the result is known:

- Run ID and scenario name
- Master seed
- Data source or source snapshot
- Workload profile
- Comparison method
- Economic rules and cost assumptions
- Run length and disturbance schedule
- Expected outputs such as JSON, HTML, manifest, screenshots, or video

> **The test must improve behavior, not improve its score by quietly changing the rules afterward.**

6. How matching works

The comparison only matters if the baseline controller and Logic Engine receive the same conditions.

- Same seed

- Same source snapshot or modeled input feed
- Same run length
- Same queue and capacity limits
- Same recovery window
- Same cost assumptions

This matched-condition rule prevents the baseline from receiving an easy workload while Logic Engine receives a different one.

7. Code examples

The images below are simple extracts showing what the installed workflow looks like. They are examples of structure and method, not a full source-code release.

Example 1 — Public data source configuration

Example 1 — Public data source configuration PYTHON

```python
PUBLIC_DATA_SOURCES = {
    "bitcoin_fees": "https://mempool.space/api/v1/fees/recommended",
    "mempool_status": "https://mempool.space/api/mempool",
    "btc_price_usd": "https://api.coingecko.com/api/v3/simple/price?ids=bitcoin&vs_
    "github_status": "https://www.githubstatus.com/api/v2/status.json",
    "cloudflare_status": "https://www.cloudflarestatus.com/api/v2/status.json",
}

def fetch_public_inputs(session):
    return {
        "fees": session.get(PUBLIC_DATA_SOURCES["bitcoin_fees"]).json(),
        "mempool": session.get(PUBLIC_DATA_SOURCES["mempool_status"]).json(),
        "price": session.get(PUBLIC_DATA_SOURCES["btc_price_usd"]).json(),
    }
```

This example shows how public endpoints are named and fetched. The point is provenance: a reviewer can see the source and understand what was observed.

Example 2 — Frozen run manifest

Example 2 — Frozen run manifest JSON

```json
{
  "run_id": "PROGRAM_40_PUBLIC_INCIDENT",
  "master_seed": 41037,
  "live_fetch_required": true,
  "source_snapshot_refs": [
    "github_status_2026-07-17T19:42:16Z.json",
    "cloudflare_status_2026-07-17T19:42:16Z.json"
  ],
  "workload_profile": "public_incident_medium_financial",
  "comparison_mode": "matched_baseline_vs_logic_engine",
  "economic_rules": "frozen_before_run",
  "report_outputs": ["json", "html", "manifest", "video"]
}
```

The manifest captures the run identity, master seed, source references, workload profile, comparison mode, and expected outputs.

Example 3 — Matched comparison rule

Example 3 — Matched comparison rule PYTHON

```python
master_seed = 41037

baseline = build_controller(
    name="strong_baseline",
    seed=master_seed,
    feed_snapshot=source_snapshot,
    rules=frozen_rules
)

logic_engine = build_controller(
    name="logic_engine",
    seed=master_seed,
    feed_snapshot=source_snapshot,
    rules=frozen_rules
)

assert baseline.input_hash == logic_engine.input_hash
```

This extract shows the central rule: the baseline and Logic Engine must receive the same seed and the same frozen input feed.

8. What evidence gets preserved

A good evidence package should preserve more than the final chart.

- Source names or source URLs
- Timestamps
- Source snapshots or profile descriptors
- Run manifest
- Seed and scenario name
- Raw logs and JSON exports
- HTML report
- Manifest and hashes
- Screenshots and screen recordings where available

9. What changes during a real customer pilot

In a real customer pilot, the public or modeled source is replaced with authorized customer data. The same reproducibility idea remains, but the environment becomes customer-specific.

Stage	Data situation
Before customer pilot	Public sources or controlled modeled workloads
During customer pilot	Authorized historical or read-only live customer telemetry
What stays the same	Frozen scope, logging, baseline comparison, measurement rules, and preserved evidence

10. One-minute explanation

> **Glyph Systems does not ask people to trust a dashboard by itself. Every serious run records where the information came from, when it was obtained, which configuration and seed were used, and whether the data was public, controlled, or customer-supplied. Logic Engine and the comparison system receive the same conditions. The raw inputs, logs, reports, manifests, hashes, screenshots, and videos are preserved so a reviewer can trace the result back to its source.**

Conclusion

The purpose of provenance is not decoration. It is trust. A result becomes more believable when the source, the seed, the frozen rules, and the evidence chain are visible.

For Glyph Systems, the rule is simple: every important run should show where the information came from, what was frozen before the run began, and what files allow someone else to inspect the result afterward.

> **Simple version: show the source, freeze the rules, preserve the outputs.**

This overview is for explanation only. Exact customer data handling, logging, retention, validation, and confidentiality controls will depend on the final commercial deployment and legal review.